User Service: Address Module Specification

Complete Database Schema, DTOs, Endpoints, Request/Response Payload & Architecture

Microservice: User Service | **Database:** PostgreSQL | **Tech Stack:** Java 21, Spring Boot 3

Authentication: Bearer JWT (Auth Service) | **Total Tables:** 1 (address) | **Total Endpoints:** 6 REST APIs

1. Overview & Architecture Role

In our Distributed E-Commerce Microservices Architecture, the **User Service** manages user profiles and shipping/billing addresses. When an order is placed in the **Order Service**, the address details are either passed explicitly or fetched from this module via OpenFeign/REST.

> **Key Design Decision:** The `user_id` column in the `address` table references the `id` from the `users` table (managed by `Auth Service`). In microservices pattern, cross-database Foreign Keys are avoided; instead, `user_id` is stored as an indexed logical reference passed inside the JWT Token header (e.g., `X-User-Id` injected by API Gateway).

2. PostgreSQL Database Schema (Table: `address`)

Below is the detailed PostgreSQL DDL and column dictionary for the `address` table.

Column Name	Data Type	Constraints	Description
`id`	BIGSERIAL / BIGINT	PRIMARY KEY	Unique address identifier (Auto Increment)
`user_id`	BIGINT	NOT NULL, INDEXED	Logical foreign key pointing to Auth Service User ID
`full_name`	VARCHAR(100)	NOT NULL	Recipient name for delivery
`phone_number`	VARCHAR(15)	NOT NULL	Contact phone number for delivery driver
`flat_house_no`	VARCHAR(150)	NOT NULL	Flat, House No, Building, Apartment name
`area_street`	VARCHAR(255)	NOT NULL	Street, Sector, Area, Village name
`landmark`	VARCHAR(150)	NULLABLE	Nearby landmark (e.g., Near Metro Station)
`city`	VARCHAR(100)	NOT NULL	City / Town
`state`	VARCHAR(100)	NOT NULL	State / Province
`pincode`	VARCHAR(10)	NOT NULL	Postal Zip/Pincode (e.g., 208001)
`address_type`	VARCHAR(20)	NOT NULL	HOME, WORK, or OTHER (Enum)
`is_default`	BOOLEAN	DEFAULT FALSE	Flag to mark primary shipping address
`created_at`	TIMESTAMP	NOT NULL	Record creation timestamp
`updated_at`	TIMESTAMP	NOT NULL	Record update timestamp

PostgreSQL DDL Script:

```
CREATE TABLE address (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    flat_house_no VARCHAR(150) NOT NULL,
    area_street VARCHAR(255) NOT NULL,
    landmark VARCHAR(150),
    city VARCHAR(100) NOT NULL,
    state VARCHAR(100) NOT NULL,
    pincode VARCHAR(10) NOT NULL,
    address_type VARCHAR(20) NOT NULL DEFAULT 'HOME',
    is_default BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_address_user_id ON address(user_id);
```

3. Complete API Endpoints Summary

Base URL Path: `/api/v1/users/addresses` (routed through API Gateway)

Method	Endpoint	Action	Success Code
POST	`/api/v1/users/addresses`	Add new address for current user	201 Created
GET	`/api/v1/users/addresses`	Get all addresses of logged-in user	200 OK
GET	`/api/v1/users/addresses/{id}`	Get single address details by ID	200 OK
PUT	`/api/v1/users/addresses/{id}`	Update an existing address	200 OK
DELETE	`/api/v1/users/addresses/{id}`	Delete address by ID	200 OK / 204 No Content
PUT	`/api/v1/users/addresses/{id}/default`	Set address as primary default	200 OK

4. Detailed API Specifications & Payloads

> **Global Response Wrapper Standard:** All API responses follow a uniform wrapper format: `{ "success": boolean, "message": string, "data": Object, "timestamp": string }`

POST `/api/v1/users/addresses`

Description: Adds a new address for the authenticated user (User ID extracted from Authorization header / Gateway context).

Request Headers:

```
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
X-User-Id: 101
```

Request Body (JSON):

```
{
  "fullName": "Vivek Yadav",
  "phoneNumber": "9876543210",
  "flatHouseNo": "Flat 302, Royal Palms",
  "areaStreet": "Civil Lines, Near Mall Road",
  "landmark": "Opposite City Hospital",
  "city": "Kanpur",
  "state": "Uttar Pradesh",
  "pincode": "208001",
  "addressType": "HOME",
  "isDefault": true
}
```

Response Body (201 Created):

```
{
  "success": true,
  "message": "Address added successfully",
  "data": {
    "id": 1,
    "userId": 101,
    "fullName": "Vivek Yadav",
    "phoneNumber": "9876543210",
    "flatHouseNo": "Flat 302, Royal Palms",
    "areaStreet": "Civil Lines, Near Mall Road",
    "landmark": "Opposite City Hospital",
    "city": "Kanpur",
    "state": "Uttar Pradesh",
    "pincode": "208001",
    "addressType": "HOME",
    "isDefault": true,
    "createdAt": "2026-07-31T10:15:30",
    "updatedAt": "2026-07-31T10:15:30"
  },
  "timestamp": "2026-07-31T10:15:30"
}
```

GET `/api/v1/users/addresses`

Description: Fetches all addresses associated with the current user.

Response Body (200 OK):


```
{
  "success": true,
  "message": "Addresses retrieved successfully",
  "data": [
    {
      "id": 1,
      "userId": 101,
      "fullName": "Vivek Yadav",
      "phoneNumber": "9876543210",
      "flatHouseNo": "Flat 302, Royal Palms",
      "areaStreet": "Civil Lines, Near Mall Road",
      "landmark": "Opposite City Hospital",
      "city": "Kanpur",
      "state": "Uttar Pradesh",
      "pincode": "208001",
      "addressType": "HOME",
      "isDefault": true,
      "createdAt": "2026-07-31T10:15:30",
      "updatedAt": "2026-07-31T10:15:30"
    },
    {
      "id": 2,
      "userId": 101,
      "fullName": "Vivek Yadav",
      "phoneNumber": "9876543210",
      "flatHouseNo": "Tech Park Tower B, 4th Floor",
      "areaStreet": "Sector 62",
      "landmark": "Near Metro Station",
      "city": "Noida",
      "state": "Uttar Pradesh",
      "pincode": "201301",
      "addressType": "WORK",
      "isDefault": false,
      "createdAt": "2026-07-31T11:00:00",
      "updatedAt": "2026-07-31T11:00:00"
    }
  ],
  "timestamp": "2026-07-31T12:00:00"
}
```

GET `/api/v1/users/addresses/{id}`

Description: Fetches address details for a specific Address ID (Validates if it belongs to logged-in user).

Response Body (200 OK):


```
{
  "success": true,
  "message": "Address details fetched",
  "data": {
    "id": 1,
    "userId": 101,
    "fullName": "Vivek Yadav",
    "phoneNumber": "9876543210",
    "flatHouseNo": "Flat 302, Royal Palms",
    "areaStreet": "Civil Lines, Near Mall Road",
    "landmark": "Opposite City Hospital",
    "city": "Kanpur",
    "state": "Uttar Pradesh",
    "pincode": "208001",
    "addressType": "HOME",
    "isDefault": true,
    "createdAt": "2026-07-31T10:15:30",
    "updatedAt": "2026-07-31T10:15:30"
  },
  "timestamp": "2026-07-31T12:05:00"
}
```

PUT `/api/v1/users/addresses/{id}`

Description: Updates fields of an existing address by ID.

Request Body (JSON):

```
{
  "fullName": "Vivek R. Yadav",
  "phoneNumber": "9876543210",
  "flatHouseNo": "Flat 302, Block B, Royal Palms",
  "areaStreet": "Civil Lines, Near Mall Road",
  "landmark": "Opposite City Hospital",
  "city": "Kanpur",
  "state": "Uttar Pradesh",
  "pincode": "208001",
  "addressType": "HOME",
  "isDefault": true
}
```

Response Body (200 OK):

```
{
  "success": true,
  "message": "Address updated successfully",
  "data": {
    "id": 1,
    "userId": 101,
    "fullName": "Vivek R. Yadav",
    "phoneNumber": "9876543210",
    "flatHouseNo": "Flat 302, Block B, Royal Palms",
    "areaStreet": "Civil Lines, Near Mall Road",
    "landmark": "Opposite City Hospital",
    "city": "Kanpur",
    "state": "Uttar Pradesh",
    "pincode": "208001",
    "addressType": "HOME",
    "isDefault": true,
    "createdAt": "2026-07-31T10:15:30",
    "updatedAt": "2026-07-31T12:10:00"
  },
  "timestamp": "2026-07-31T12:10:00"
}
```

PUT `/api/v1/users/addresses/{id}/default`

Description: Sets specified address as default and automatically sets `is_default = false` for all other addresses of that user.

Response Body (200 OK):

```
{
  "success": true,
  "message": "Set as default address successfully",
  "data": { "id": 2, "isDefault": true },
  "timestamp": "2026-07-31T12:15:00"
}
```

DELETE `/api/v1/users/addresses/{id}`

Description: Deletes address permanently by ID.

Response Body (200 OK):

```
{
  "success": true,
  "message": "Address deleted successfully",
  "data": null,
  "timestamp": "2026-07-31T12:20:00"
}
```

5. Error Handling & Validation Responses

When input validation fails (e.g. invalid pincode or blank required fields) or address is not found:

400 Bad Request (Validation Error):

```
{
  "success": false,
  "message": "Validation Failed",
  "errors": {
    "pincode": "Pincode must be exactly 6 digits",
    "phoneNumber": "Phone number is required"
  },
  "timestamp": "2026-07-31T12:25:00"
}
```

404 Not Found (Address Not Found):

```
{
  "success": false,
  "message": "Address with ID 999 not found for this user",
  "data": null,
  "timestamp": "2026-07-31T12:26:00"
}
```

6. Spring Boot 3 Implementation Blueprint

1. JPA Entity (`Address.java`)

```
@Entity
@Table(name = "address")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Address {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id", nullable = false)
    private Long userId;

    @Column(name = "full_name", nullable = false, length = 100)
    private String fullName;

    @Column(name = "phone_number", nullable = false, length = 15)
    private String phoneNumber;

    @Column(name = "flat_house_no", nullable = false, length = 150)
    private String flatHouseNo;

    @Column(name = "area_street", nullable = false, length = 255)
    private String areaStreet;

    private String landmark;

    @Column(nullable = false, length = 100)
    private String city;

    @Column(nullable = false, length = 100)
    private String state;

    @Column(nullable = false, length = 10)
    private String pincode;

    @Enumerated(EnumType.STRING)
    @Column(name = "address_type", nullable = false)
    private AddressType addressType; // HOME, WORK, OTHER

    @Column(name = "is_default", nullable = false)
    private Boolean isDefault;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;
}
```

2. Spring Data JPA Repository (`AddressRepository.java`)

```
@Repository
public interface AddressRepository extends JpaRepository<Address, Long> {

    List<Address> findByUserId(Long userId);

    Optional<Address> findByIdAndUserId(Long id, Long userId);

    @Modifying
    @Query("UPDATE Address a SET a.isDefault = false WHERE a.userId = :userId")
    void resetDefaultAddressForUser(Long userId);
}
```

3. Request DTO with Jakarta Validations (`AddressRequestDto.java`)

```
@Data
public class AddressRequestDto {

    @NotBlank(message = "Full name is required")
    @Size(max = 100, message = "Full name cannot exceed 100 characters")
    private String fullName;

    @NotBlank(message = "Phone number is required")
    @Pattern(regexp = "^[0-9]{10}$", message = "Phone number must be valid 10 digits")
    private String phoneNumber;

    @NotBlank(message = "Flat / House No is required")
    private String flatHouseNo;

    @NotBlank(message = "Area / Street is required")
    private String areaStreet;

    private String landmark;

    @NotBlank(message = "City is required")
    private String city;

    @NotBlank(message = "State is required")
    private String state;

    @NotBlank(message = "Pincode is required")
    @Pattern(regexp = "^[0-9]{6}$", message = "Pincode must be 6 numeric digits")
    private String pincode;

    private AddressType addressType = AddressType.HOME;

    private Boolean isDefault = false;
}
```

4. REST Controller (`AddressController.java`)

```
@RestController
@RequestMapping("/api/v1/users/addresses")
@RequiredArgsConstructor
public class AddressController {

    private final AddressService addressService;

    @PostMapping
    public ResponseEntity<ApiResponse<AddressResponseDto>> addAddress(
            @RequestHeader("X-User-Id") Long userId,
            @Valid @RequestBody AddressRequestDto request) {
```

```
        AddressResponseDto response = addressService.addAddress(userId, request);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(ApiResponse.success("Address added successfully", response));
    }

    @GetMapping
    public ResponseEntity<ApiResponse<List<AddressResponseDto>>> getUserAddresses(
            @RequestHeader("X-User-Id") Long userId) {
        List<AddressResponseDto> list = addressService.getAddressesByUserId(userId);
        return ResponseEntity.ok(ApiResponse.success("Addresses retrieved successfully", list));
    }

    @GetMapping("/{id}")
    public ResponseEntity<ApiResponse<AddressResponseDto>> getAddressById(
            @RequestHeader("X-User-Id") Long userId,
            @PathVariable Long id) {
        AddressResponseDto response = addressService.getAddressById(userId, id);
        return ResponseEntity.ok(ApiResponse.success("Address details fetched", response));
    }

    @PutMapping("/{id}")
    public ResponseEntity<ApiResponse<AddressResponseDto>> updateAddress(
            @RequestHeader("X-User-Id") Long userId,
            @PathVariable Long id,
            @Valid @RequestBody AddressRequestDto request) {
        AddressResponseDto response = addressService.updateAddress(userId, id, request);
        return ResponseEntity.ok(ApiResponse.success("Address updated successfully", response));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<ApiResponse<Void>> deleteAddress(
            @RequestHeader("X-User-Id") Long userId,
            @PathVariable Long id) {
        addressService.deleteAddress(userId, id);
        return ResponseEntity.ok(ApiResponse.success("Address deleted successfully", null));
    }

    @PutMapping("/{id}/default")
    public ResponseEntity<ApiResponse<Void>> setDefaultAddress(
            @RequestHeader("X-User-Id") Long userId,
            @PathVariable Long id) {
        addressService.setDefaultAddress(userId, id);
        return ResponseEntity.ok(ApiResponse.success("Set as default address successfully", null));
    }
}
```